

```

LeeCadena(A);
LeeCadena(B);
Concatena(A,B);
Writeln('');
EscribeCadena(A);
End.

```

Se aprecia en la función **Concatena** que el hecho de poder acceder a la longitud de la cadena permite escribir algoritmos muy eficientes de cadenas de caracteres. Sin embargo, esta estructura definida para las cadenas de caracteres es ineficiente en cuanto almacenamiento. Se están almacenando los caracteres ASCII en 2 bytes mientras que sería suficiente uno sólo.

9.5 Registros

La situación encontrada en el problema planteado anteriormente de manipulación de cadenas es muy común. Lo más frecuente es encontrarse en una situación en la que se quiere representar en un tipo de dato una información que posee estructura interna y es heterogénea, es decir, los campos en los que se subdivide el dato no son todos del mismo tipo. Para ello el *PASCAL*, como otros lenguajes de programación actuales, suministra los datos tipo registro (en inglés *records*).

Un registro es un tipo de dato definido por el programador en el que puede especificar su estructura interna.

El programador da nombre al nuevo tipo de dato y a cada uno de los campos que lo componen, y especifica el tipo de dato que puede ocupar cada uno de los campos. La sintaxis para estas especificaciones es:

```

Type
    nombre =
        Record
            NombreCampo1 : TipoDato1 ;
            NombreCampo2 : TipoDato2 ;
            . . . . .
        End;

```

donde *nombre* es el identificador elegido para el registro, y *NombreCampo1*, *NombreCampo2*, ... son los identificadores elegidos para los distintos

campos. *TipoDato1* , *TipoDato2*, ... son la especificación del tipo de dato que se va a almacenar en cada uno de los campos. Esta especificación puede corresponder a un tipo de dato estandar del *PASCAL* , la enumeración o el rango de los posibles valores, u otro tipo de dato declarado con anterioridad.

Por ejemplo, hay casos en los que puede ser conveniente definir un tipo de variable para almacenar la información de una fecha. Una posible definición es la siguiente:

```

Program Seis;
  Type
    Fecha =
      Record
        mes : 0 .. 12; { 0 seria para indicar que no se conoce la fecha}
        dia : 1 .. 31;
        agno : Integer ;
      End;

  Var
    alta , baja : Fecha;

```

El modo en el que se especifica refiriéndose a una variable del tipo **Record** un determinado campo, es añadiendo al nombre de la variable el nombre dado al campo, y unidos por un punto. El ejemplo anterior podría continuar del siguiente modo

```

Begin
  alta.dia := 27 ;
  alta.mes := 2 ;
  alta.agno := 1992;
  baja.mes := alta.mes + 2;
  ....

```

También el *PASCAL* permite la comodidad de la estructura **With** para acceder a los valores de los campos de los registros. La sintaxis de la estructura **With** es la siguiente :

With *nombre* **Do** *accion*

donde *nombre* es la especificación de la variable (o las variables separadas por comas) del tipo **Record** a la que se refiere la instrucción *PASCAL accion*

que se encuentran a continuación del **Do**. Si se trata de una instrucción compuesta se encontrará, como siempre, horquillada entre un **Begin** y un **End**. En la instrucción del **With** podemos especificar los distintos datos del registro con sólo el nombre de los campos. Por ejemplo, las instrucciones anteriores, se podrían haber escrito del siguiente modo:

```

Begin
  With alta Do
    Begin
      dia := 27 ;
      mes := 2  ;
      agno := 1992;
      baja.mes := mes + 2;
    End;
  ....

```

La elección del uso de la estructura **With** se suele hacer en cada caso particular según la legibilidad que añada al programa.

Ya hemos dicho que los campos de los registros pueden contener datos que son del tipo registro. En ese caso, como siempre, habrá que llevar cuidado en definir los tipos de datos en el orden adecuado para que nunca aparezca en una declaración un tipo de dato que no ha sido declarado en pasajes anteriores del programa. Si ampliamos el ejemplo anterior hacia la construcción de una base de datos con los alumnos y sus notas, un programa de captación de datos podría empezar así:

```

Program Siete;
Type
  Fecha =
    Record
      mes : 0 .. 12; { 0 seria para indicar que no se conoce la fecha}
      dia : 1 .. 31;
      agno : Integer ;
    End;
  Alumno =
    Record
      nombre : String;
      apellidos : String;

```

```

        nacimiento : Fecha;
        nota : Real;
    End;
Var
    uno, otro : Alumno ;

Begin
    Writeln('Nombre : ') ; Readln(uno.nombre);
    Writeln('Apellidos : '); Readln(uno.apellidos);
    Writeln('Año de nacimiento : '); Readln(uno.nacimiento.agno);
    Writeln('Mes de nacimiento: '); Readln(uno.nacimiento.mes);
    Writeln('Día de nacimiento: '); Readln(uno.nacimiento.dia);
    Writeln('Nota del examen: '); Readln(uno.nota);
    . . . . .

```

y vemos que la referencia a los campos de registros que son a su vez campos de un registro se realiza concatenando con puntos los identificadores de los campos.

Para crear una base de datos que contuviera información de los alumnos de una clase lo lógico sería ordenar los alumnos en una lista. Para ello, se puede crear una estructura **Array** cuyos elementos sean los registros definidos para almacenar la información de los alumnos. Se podría por ejemplo añadir en la parte declarativa del programa **Siete** el siguiente tipo de dato:

```

{ Type }
    Lista =
        Array[1..100] of alumno;

```

y también una variable de este tipo:

```

{ Var }
    primero : Lista;

```

El acceso a los elementos del **Array** es el habitual. Por ejemplo, el programa **Siete** podría continuar del siguiente modo:

```

    primero[1] := uno;
    If (Primero[1].nacimiento.mes = 12 ) Then .....

```

En la primera de estas instrucciones se asigna al elemento primero del **Array primero** el dato almacenado en **uno** (con todos sus campos de una sola vez). En la segunda, se comprueba el valor que tiene el dato almacenado en el campo **mes** del campo **nacimiento** de la variable del tipo **Alumno** almacenado en la posición primera del **Array primero**.

También existe, obviamente, la posibilidad inversa: la de definir registros en los que sus campos sean datos con la estructura **Array**. Podemos retomar ahora el problema estudiado anteriormente de cadenas de caracteres y utilizando registros disponer de una estructura de datos con aprovechamiento eficiente de la memoria. El tipo de dato **Cadena** se define ahora como un registro con un campo que es el número de caracteres almacenados y el otro campo el **Array** de caracteres. Así, para la longitud utilizamos un **Integer** y para cada carácter un sólo byte.

```

Program Ocho;
Const
  MAXDIM = 300;
Type
  Ristra = Array [1..MAXDIM] of Char;
  Cadena =
    Record
      longitud  : Integer;
      contenido : Ristra;
    End;
Var
  A,B : Cadena;
Procedure LeeCadena (Var x : Cadena);
Var
  i  : Integer;
  c  : Char;
Begin { LeeCadena }
  i:= 1;
  With x Do
  Begin
    Repeat
      Read(c);
      contenido[i] := c;
      i := i+1;
    
```

```

    Until ( EOLN );
    longitud := i - 1;
End;
{ Lee los caracteres ASCII 13 (CR) y 10 (LF)
  que delimitan el fin de linea }
Read(c);Read(c);
End; { LeeCadena }

Procedure EscribeCadena ( x : Cadena);
Var
  i: Integer;
Begin { EscribeCadena }
  For i := 1 to x.longitud Do Write(x.contenido[i] );
End; { EscribeCadena }

{ Procedimiento para unir a la cadena A1 la cadena A2 }
Procedure Concatena ( Var
                      A1 , A2 : Cadena );

Var
  dim1 , i : Integer;
Begin
  dim1 := A1.longitud;
  For i := 1 To A2.longitud Do
    A1.contenido[ dim1 + i ] := A2.contenido[i];
  A1.longitud := dim1 + A2.longitud;
End;

Begin
  Writeln('');
  LeeCadena(A);
  LeeCadena(B);
  Concatena(A,B);
  Writeln('');
  EscribeCadena(A);
End.

```

Después de estudiar todos estos tipos de datos definidos por el usuario

debe estar mucho más claro el sentido de los conceptos *tipo de dato* y *variable*. Una *variable*, es un identificador que utiliza el programador para referirse a un dato y poder realizar operaciones con él. El *tipo de dato* ha de especificarse para que el compilador pueda generar código UCP en el que se utilice una cantidad de memoria suficiente para ese dato y estructurada del modo adecuado. Cuando el programador se enfrenta a la resolución de un problema puede pensar en variables para cualquier concepto abstracto que piense que sea útil para resolver el problema de un modo claro y comunicable, lo que muchas veces quiere decir de un modo lo más próximo posible al lenguaje natural. La restricción obvia es poder explicitar sin ambigüedades la gestión de la memoria del ordenador que ha de realizar el compilador para almacenar y manipular ese dato. Mediante la estructura **Record** y **Array** el *PASCAL* ofrece la posibilidad de definir tipos de datos muy próximos a los utilizados en el lenguaje natural y que son una organización precisa de tipos de datos más sencillos. En última instancia, los átomos que van a formar esas estructuras más complejas son los tipos de datos fundamentales del *PASCAL* : **Byte** , **Integer** , Al final, para una variable, el compilador reservará lugar en memoria para almacenar un número determinado de bits de información (una sucesión de ceros o unos). El tipo de dato definido para esa variable va a determinar el modo en el que esos bits van a intervenir en las operaciones y también el modo con el que el programador se podrá referir a todos esos bits de golpe o a subconjuntos de ellos.

9.6 Uniones

Comprendido lo anterior no debe resultar difícil entender dos posibilidades avanzadas que permite el *PASCAL* en la manipulación de registros, y que son los registros con variante (o uniones con discriminación) y las uniones libres. En *PASCAL* es posible dar una definición dinámica de la composición de un registro. Es decir, que los campos que lo componen varíen según el valor de un parámetro. Para ello se utiliza la siguiente modificación de la estructura **Case** dentro de la definición del registro:

Case

```

nombre : TipodeDato Of
  caso1 : ( Especificacion1 );
  caso2 : ( Especificacion2 );
  . . . . .

```

donde *nombre* es el identificador elegido para la variable cuyo valor determina la composición del registro, *TipodeDato* el tipo de dato de la variable *nombre*, *caso1*, *caso2*, ... los valores o rango de valores que darán lugar a las distintas especificaciones del contenido del registro *Especificacion1*, *Especificacion2*,... La enumeración de las opciones no acaba con un **End**; por que existe la restricción de que las variantes se han de colocar en la última parte de la definición de un registro y por tanto acaban con el **End**; del fin de la definición del registro.

Como ejemplo de la utilización de los registros con variante vamos a considerar el caso visto anteriormente de la base de datos de alumnos. Supongamos que en el campo de la nota podemos querer guardar en alguna ocasión en vez de un dato **Real** una calificación global como **Aprobado** y elegimos para ello un dato **String**. Una posibilidad es la siguiente modificación del caso anterior:

```

Program Nueve;
  Type
    Fecha =
      Record
        mes : 0 .. 12; { 0 seria para indicar que no se conoce la fecha}
        dia : 1 .. 31;
        agno : Integer ;
      End;
    Alumno =
      Record
        nombre : String;
        apellidos : String;
        nacimiento : Fecha;
        Case final : Boolean Of
          False : ( nota : Real );
          True : ( notaFinal : Real;
                  calificacion : String[14] );
        End;
  Var
    uno, otro : Alumno ;
  Begin
    .....

```

Instrucciones válidas de este programa serían tanto

```
uno.nota:= 6.8 ;
```

como,

```
uno.calificacion := 'Notable';
```

y al programador le queda toda la responsabilidad de utilizar una u otra. Realmente, esta estructura del *PASCAL* no es muy feliz en el sentido de que el compilador no verifica el valor de la variable que da acceso a las variaciones en los campos y su figura es meramente recordatoria para el programador. Sea cual sea el valor de la variable utilizada para la bifurcación del tipo de dato, se puede acceder a la información almacenada en el registro con cualquiera de las variantes. Es responsabilidad exclusiva de programador utilizar cada campo cuando está definido. Con las **Uniones libres** se hace más explícito que la utilización de los campos alternativos es pura responsabilidad del programador y no se indica identificador para la variable que gobierna las opciones. Para definir una **Union Libre** dentro de la definición de la estructura **Record** se especifica la alternativa del modo siguiente:

Case

TipodeDato Of

```
caso1 : ( Especificacion1 );
```

```
caso2 : ( Especificacion2 );
```

```
. . .
```

donde *caso1*, *caso2*,... son valores posibles del tipo de dato especificado *TipodeDato*. Es equivalente a un Registro con variante en el que se omite el identificador de la variable que gobierna las distintas alternativas.

Esta posibilidad de acceso variable al contenido de un registro es coherente con lo resumido anteriormente sobre el sentido de variables y tipos de datos. El compilador reserva para el dato en la memoria un espacio suficiente para almacenar aquella de las variantes del registro de mayor tamaño. Se rellenará el espacio de memoria del modo que en cada momento desee el programador y la elección se realiza usando un campo u otro. Igualmente, la interpretación de los bits almacenados en esas posiciones de memoria también depende del campo del registro que se utilice, en el caso de que distintos campos correspondan a distintos tipos de datos.

Las **Uniones Libres** han de usarse con precaución para evitar confusiones en la lectura de los programas, pero a veces son las adecuadas para hacer un programa legible. En el siguiente ejemplo la unión entre el tipo de dato **Char** y **Byte** se utiliza para escribir una función que convierte un carácter en el correspondiente número de orden ASCII.

```

Program Diez;

Var
  dato : Char;

Function ElAscii ( x : Char ) : Byte;
Type
  Atomo =
    { Union libre de un Char con un Byte }
    Record
      Case Integer Of
        1 : ( car : Char);
        2 : ( num : Byte );
      End;
Var
  y : Atomo;
Begin
  y.car := x;
  ElAscii := y.num
End;

Begin { Diez }
  Readln (dato);
  Writeln ( ' El caracter ',dato , ' corresponde al ASCII numero : ',
            ElAscii(dato) );
End. { Diez }

```

En la función **ElAscii** el tipo de dato **Atomo** se define como la unión libre de un **Char** y un **Byte**. En este caso, ambos tipos de datos ocupan el mismo espacio en memoria (un byte). La variable **y** ocupa pues un byte

de memoria, pero la sucesión de bits que lo componen puede interpretarse o bien como un **Char** o un **Byte**, dependiendo de como se referencie.